

Partiel Graphes et Optimisation Combinatoire

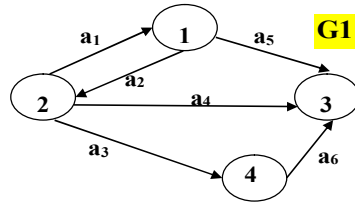
Exercice 1 (3.5 Pts=1.0+1.0+1.5)

- 1) Représenter le graphe orienté G1 ci-contre par sa :

- a) Matrice d'incidence.
b) Matrice d'incidence ligne par ligne.

- 2) **Ecrire en C**, pour chacune des représentations ci-dessus, les structures de données nécessaires à la représentation d'un graphe.

- 3) **Ecrire en C**, le passage d'un graphe orienté $G=(S, A)$ représenté par sa par **matrice d'incidence ligne par ligne** à une autre représentation par **matrice d'incidence**.

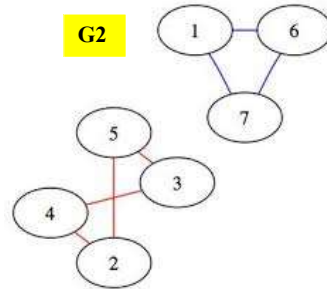


Exercice 2 (4.25 Pts=0.25+2.25+0.75)

- 1) **Ecrire en C**, les structures de données nécessaires à la représentation d'un **graphe orienté par une matrice d'adjacence**.

- 2) Soit $G=(S, A)$ un graphe non orienté représenté par sa matrice d'adjacence. En utilisant le **principe des parcours en largeurs** ; écrire en langage algorithmique (ou en C), un algorithme qui détermine les **composantes connexes** ainsi que leurs nombres présent dans G. Pour les composantes connexes, on utilise un tableau CC de taille $n=Card(S)$ indicé par les sommets s du graphe tel que $CC[s]$ contient le numéro de la composante connexe.

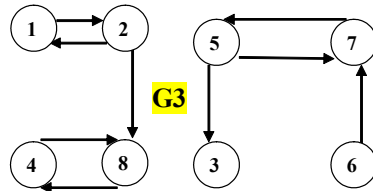
- 3) Dérouler votre algorithme au graphe G2 ci-contre.



Exercice 3 (4.25 Points=1.25+0.75+2.25)

- 1) Appliquer étape par étape la méthode des deux parcours en profondeur pour trouver les composantes fortement connexes de **G3**

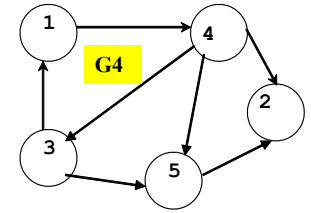
- 2) **Ecrire en C**, les structures de données nécessaires à la représentation d'un **graphe orienté par une liste des successeurs et des prédécesseurs**.



- 3) Soit $G=(S, A)$ un graphe orienté représenté par une **liste des successeurs et des prédécesseurs** ; écrire en langage algorithmique (ou en C), un algorithme qui détermine les **composantes fortement connexes** ainsi que leurs nombres présent dans G. Pour les composantes fortement connexes, on utilise un tableau CFC de taille $n=Card(S)$ indicé par les sommets s du graphe tel que $CFC[s]$ contient le numéro de la composante fortement connexe.

Exercice 4 (4.0 Pts=0.5+2.0+1.5)

- 1) **Ecrire en C**, les structures de données nécessaires à la représentation d'un **graphe orienté par une liste d'adjacence**.
2) Si on suppose que G est un **graphe orienté** représenté par sa **liste d'adjacence** ; **écrire en C**, un algorithme qui détermine la fermeture transitive de G.
3) Appliquer étape par étape l'algorithme de la fermeture transitive au graphe G4 ci-contre.



Exercice 5 (4.0 Pts=0.5+2.5+1.0)

- 1) **Ecrire en C**, les structures de données nécessaires à la représentation d'un **graphe orienté par une file des successeurs**.

APS	1	2	2	3	5	8
-----	---	---	---	---	---	---

- 2) Si on suppose que G est un graphe orienté représenté par sa **file des successeurs** ; écrire en langage **File des Successeurs** algorithmique (ou en C), un algorithme qui vérifie la **présence d'un circuit** dans un graphe G donné.

FS	4	1	2	3	2	3	4
----	---	---	---	---	---	---	---

- 3) Appliquer étape par étape votre algorithme au graphe G5 ci-dessus.

Notes : on suppose connu les algorithmes :

- Gestion des tableaux et des matrices :
initTab, allocTab, libTab, initMat, allocMat, libMat...
- Primitives des différents membres des structures.
- Allocations et libérations des structures :
allocSMA, libSMA, ... // Structure matrice d'adjacence
allocSMI, libSMI, ... // Structure matrice d'incidence
allocSMILL, libSMILL // Structure matrice d'incidence Ligne / Ligne
allocSLA, libSLA, ... // Structure liste d'adjacence
allocSFS, libSFS, ... // Structure file des successeurs
allocSLASP, libSLASP, ... // Structure liste des successeurs et des prédécesseurs
- Gestion des piles (initP, estVideP, sommetP, empiler, depiler, ...)
- Gestion des files (initF, estVideF, sommetF, enfiler, defiler, ...)
- Gestion des listes chaînées (initL, insertL, suppL, estVideL, ...)
- Gestion des ensembles (initE, insertE, suppE, estVideE, ...)

Veuillez donc expliciter les prototypes de ses algorithmes au cas où vous les utiliser.